
Applied Reinforcement Learning

Dynamic Programming Exercises

Riccardo Berta

2026.09.17

0.1 Exercises on Dynamic Programming

These activities reinforce the core ideas presented in the DP chapter: the Bellman expectation and optimality equations, exact policy evaluation by solving a linear system, iterative policy evaluation as a fixed-point iteration, the policy-improvement theorem, value iteration, the existence of multiple optimal policies, the computational efficiency of policy iteration versus brute-force search and versus value iteration, and how discounting interacts with risk in stochastic environments.

0.1.1 Exercise 3.1 — Exact policy evaluation

Consider the following MDP with two non-terminal states S_1, S_2 and one terminal state, evaluated under a fixed policy (one action per state), with $\gamma = 1$:

- In S_1 : the action leads deterministically to S_2 with reward -1.
- In S_2 : the action leads with probability 0.5 to the terminal state with reward +10, and with probability 0.5 back to S_1 with reward -1.

Compute the state values $v_\pi(S_1)$ and $v_\pi(S_2)$.

Step 1 — Write the Bellman expectation equation

$v_\pi(s) = \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma v_\pi(s')]]$ for each state (with $\gamma = 1$, and $v_\pi(\text{terminal}) = 0$):

$$v_\pi(S_1) = 1 \cdot [-1 + v_\pi(S_2)] = -1 + v_\pi(S_2)$$

$$v_\pi(S_2) = 0.5[10 + 0] + 0.5[-1 + v_\pi(S_1)] = 4.5 + 0.5 v_\pi(S_1)$$

Step 2 — Solve the linear system

Substitute the first equation into the second:

$$v_\pi(S_2) = 4.5 + 0.5(-1 + v_\pi(S_2)) = 4.0 + 0.5 v_\pi(S_2).$$

$$\Rightarrow 0.5 v_\pi(S_2) = 4.0 \Rightarrow v_\pi(S_2) = 8.$$

Back-substitute:

$$v_\pi(S_1) = -1 + 8 = 7.$$

Key concept

For a fixed policy the Bellman expectation equations form a linear system with a unique solution, so value functions can be computed exactly when the MDP is small and known. Iterative methods approximate this same solution when a direct solve is impractical.

0.1.2 Exercise 3.2 — Iterative policy evaluation

For the same MDP, run iterative policy evaluation starting from:

$$V_0(S_1) = 0$$

$$V_0(S_2) = 0$$

Remember that:

$$V_{k+1}(s) \leftarrow \sum_{s',r} p(s',r | s, \pi(s)) [r + \gamma V_k(s')].$$

Perform four sweeps and comment on convergence toward the exact values.

Step 1 — Update equations

$$V_{k+1}(S_1) = -1 + V_k(S_2)$$

$$V_{k+1}(S_2) = 4.5 + 0.5 V_k(S_1)$$

Step 2 — Iterate

sweep k	$V_k(S_1)$	$V_k(S_2)$
0	0	0
1	$-1 + 0 = -1.0$	$4.5 + 0.5(0) = 4.5$
2	$-1 + 4.5 = 3.5$	$4.5 + 0.5(-1) = 4.0$
3	$-1 + 4.0 = 3.0$	$4.5 + 0.5(3.5) = 6.25$
4	$-1 + 6.25 = 5.25$	$4.5 + 0.5(3.0) = 6.0$

Step 3 — Convergence

The estimates march toward the fixed point (7,8). Each sweep is a contraction toward the true value function.

Key concept

Iterative policy evaluation is bootstrapping: it updates each state's estimate from the current estimates of its successors. It converges to the same that solves the linear system, without ever inverting a matrix (the practical route when the state set is large). Remember that the index k counts algorithm sweeps, not environment time steps.

0.1.3 Exercise 3.3 — Policy improvement by one greedy step

In a state X two actions are available and both give reward 0 on the transition, with $\gamma = 0.9$:

- a_1 : leads deterministically to state Y, with known value $v_\pi(Y) = 5$
- a_2 : leads deterministically to state Z, with known value $v_\pi(Z) = 8$

The current policy selects a_1 in X.

1. Compute the action values of both actions
2. Apply a policy-improvement step and verify it satisfies the policy-improvement theorem.

Step 1 — Action values

$$q_\pi(X, a) = \sum_{s', r} p(s', r | X, a) [r + \gamma v_\pi(s')]$$

We have deterministic transitions and $r=0$, so:

$$q_\pi(X, a_1) = 0 + 0.9 \times 5 = 4.5$$

$$q_\pi(X, a_2) = 0 + 0.9 \times 8 = 7.2$$

Under the current policy, $v_\pi(X) = q_\pi(X, a_1) = 4.5$.

Step 2 — Greedy improvement

Considering the improvement formula:

$$\pi'(X) = \arg \max_a q_\pi(X, a) = a_2$$

since $7.2 > 4.5$.

Step 3 — Check the policy-improvement theorem

The theorem states that:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \forall s, \rightarrow v_{\pi'} \geq v_\pi$$

Here

$$q_\pi(X, \pi'(X)) = q_\pi(X, a_2) = 7.2 \geq v_\pi(X) = 4.5,$$

so switching to a_2 is guaranteed not to worsen (here, to strictly improve) the policy.

Key concept

Acting greedily with respect to the current value function yields a policy that is at least as good (the engine behind policy iteration).

0.1.4 Exercise 3.4 — Value iteration and policy extraction

Consider states A, B and a terminal goal G (with state value 0), $\gamma = 0.9$:

- In A: action "right" $\rightarrow$ B (reward 0); action "stay" $\rightarrow$ A (reward 0)

- In B: action "right" $\rightarrow$ G (reward 1); action "left" $\rightarrow$ A (reward 0)

Run value iteration for three iterations, then extract the greedy optimal policy.

Step 1 — Iterate.

With

$$V_0 = 0$$

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma V_k(s')],$$

Iteration 1:

$$V_1(A) = \max_{a \in \text{right, stay}} \sum_{s',r} p(s',r | A,a) [r + 0.9V_0(s')] = \max(1[0 + 0.9 \cdot 0], 1[0 + 0.9 \cdot 0]) = 0$$

$$V_1(B) = \max_{a \in \text{right, left}} \sum_{s',r} p(s',r | B,a) [r + 0.9V_0(s')] = \max(1[1 + 0.9 \cdot 0], 1[0 + 0.9 \cdot 0]) = 1$$

Iteration 2:

$$V_2(A) = \max_{a \in \text{right, stay}} \sum_{s',r} p(s',r | A,a) [r + 0.9V_1(s')] = \max(1[0 + 0.9 \cdot 1], 1[0 + 0.9 \cdot 0]) = 0.9$$

$$V_2(B) = \max_{a \in \text{right, left}} \sum_{s',r} p(s',r | B,a) [r + 0.9V_1(s')] = \max(1[1 + 0.9 \cdot 0], 1[0 + 0.9 \cdot 0]) = 1$$

Iteration 3:

$$V_3(A) = \max_{a \in \text{right, stay}} \sum_{s',r} p(s',r | A,a) [r + 0.9V_2(s')] = \max(1[0 + 0.9 \cdot 1], 1[0 + 0.9 \cdot 0.9]) = 0.9$$

$$V_3(B) = \max_{a \in \text{right, left}} \sum_{s',r} p(s',r | B,a) [r + 0.9V_2(s')] = \max(1[1 + 0.9 \cdot 0], 1[0 + 0.9 \cdot 0.9]) = 1$$

The values have converged:

$$v_*(A) = 0.9$$

$$v_*(B) = 1$$

Step 2 — Extract the greedy policy

Once the optimal value function has been obtained, we extract the optimal policy by choosing, in each state, the action that maximizes the expected return:

$$\pi_*(s) = \arg \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma v_*(s')]$$

For state A:

$$\text{right} : 0 + 0.9 v_*(B) = 0 + 0.9 \cdot 1 = 0.9$$

$$\text{stay} : 0 + 0.9 v_*(A) = 0 + 0.9 \cdot 0.9 = 0.81$$

Therefore:

$$\pi_*(A) = \text{right}.$$

For state B:

$$\text{right} : 1 + 0.9 v_*(G) = 1 + 0.9 \cdot 0 = 1$$

$$\text{left} : 0 + 0.9 v_*(A) = 0 + 0.9 \cdot 0.9 = 0.81$$

Therefore:

$$\pi_*(B) = \text{right}.$$

Thus,

$$\pi_*(A) = \text{right}$$

$$\pi_*(B) = \text{right}$$

and the optimal behavior is:

$$A \rightarrow B \rightarrow G$$

Key concept

Value iteration combines policy evaluation and policy improvement into a single update: each sweep updates the value function while simultaneously making the implied policy greedy with respect to the current values. The optimal policy is extracted only at the end, by taking the argmax over the one-step lookahead for each state.

0.1.5 Exercise 3.5 — How the Discount Factor Affects the Optimal Action

From a start state, the agent must choose between two actions. Each action produces a different sequence of rewards and eventually leads to a terminal state:

- SOONER: the agent receives a reward of +1 at the next time step, after which the episode terminates.
- LATER: the agent receives no reward at the next time step and a reward of +2 one step later, after which the episode terminates.

Determine the value of the discount factor for which the agent is indifferent between SOONER and LATER. Then determine which action is optimal for values of the discount factor below and above this threshold.

Step 1 — Compute the value of each action

Using the definition of the discounted return,

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

we obtain:

$$v(\text{SOONER}) = \gamma^0 \cdot 1 = 1$$

$$v(\text{LATER}) = \gamma^0 \cdot 0 + \gamma^1 \cdot 2 = 2\gamma$$

Step 2 — Find the indifference point

The agent is indifferent between the two actions when they produce the same return. Therefore, we set their values equal:

$$v(\text{SOONER}) = v(\text{LATER}) \Rightarrow 1 = 2\gamma$$

$$\Rightarrow \gamma = \frac{1}{2}$$

Step 3 — Determine the optimal action

We now compare the two action values on either side of the threshold:

- If $\gamma < \frac{1}{2}$, then $2\gamma < 1$, so SOONER is optimal: the delayed reward of +2 is discounted enough to be worth less than the immediate reward of +1.
- If $\gamma > \frac{1}{2}$, then $2\gamma > 1$, so LATER is optimal: the larger delayed reward outweighs the immediate reward.
- If $\gamma = \frac{1}{2}$, both actions have value 1, so the agent is indifferent between SOONER and LATER.

Key concept

The optimal policy is not determined by the rewards alone: it also depends on the discount factor. The discount factor controls how strongly the agent values delayed rewards relative to immediate ones. As a result, changing a factor can change which action is optimal. This also illustrates why discounting may favor policies that obtain rewards sooner, even when alternative policies can produce larger rewards later.

0.1.6 Exercise 3.6 — Multiple Optimal Actions and Policies

Consider a state s in which two different actions have the same optimal action-value:

$$q(s, a_1) = q(s, a_2) = 10$$

while every other action available in s satisfies

$$q_*(s, a) < 10$$

Now consider two policies, that are identical in every state except s :

$$\pi_1(s) = a_1$$

$$\pi_2(s) = a_2$$

Show that both policies are optimal policies. Then show that, although the two policies select different actions in state s , they induce the same optimal state-value function.

Step 1 — Determine the optimal value of state s

By definition, the optimal state-value is the maximum action-value over all actions available in s :

$$v_*(s) = \max_a q_*(s, a)$$

Since both actions achieve the maximum value,

$$q_*(s, a_1) = q_*(s, a_2) = 10,$$

we have

$$v_*(s) = \max_a q_*(s, a) = 10$$

Therefore, choosing either a_1 or a_2 in state s achieves the same optimal value.

Step 2 — Show that both policies are optimal

By construction, the two policies select the same actions in every state except s . In s , however, both policies select an action that achieves the optimal value:

$$q_*(s, \pi_1(s)) = q_*(s, \pi_2(s)) = v_*(s) = 10$$

Therefore, both policies select an optimal action in every state and are consequently optimal policies:

$$\pi_1 \neq \pi_2, \quad \pi_1, \pi_2 \in \Pi_*$$

Although they are different policies, they achieve the same optimal value in every state. Hence,

$$v_{\pi_1} = v_{\pi_2} = v_*$$

This illustrates an important distinction: the optimal value function is unique, but the optimal policy need not be.

Key concept

The optimal state-value function of an MDP is unique, but the optimal policy need not be. Whenever multiple actions achieve the same maximum value of the action-value function in a state, any of those actions can be selected without affecting optimality. This is why the optimal policy is written as

$$\pi_*(s) \in \arg \max_a q_*(s, a)$$

rather than using equality: the argmax may contain more than one optimal action.

0.1.7 Exercise 3.7 — Policy Iteration vs. Brute-Force Search

Consider an MDP with 10 states and 4 actions available in each state.

A brute-force approach searches for an optimal policy by evaluating every possible deterministic policy. In contrast, suppose that policy iteration starts from an arbitrary policy and reaches an optimal policy after 5 policy-improvement steps.

1. Compute the number of policies evaluated by brute-force search and by policy iteration
2. Compute the ratio and explain what this comparison illustrates about searching directly through the policy space.

Step 1 — Compute the number of policies evaluated

With $|A| = 4$ actions and $|S| = 10$ states, the total number of deterministic policies considered by brute-force search is

$$N_{\text{BF}} = |A|^{|S|} = 4^{10} = 1,048,576.$$

Policy iteration, instead, evaluates the initial policy and the $K = 5$ policies obtained through the improvement steps:

$$N_{\text{PI}} = K + 1 = 6$$

Step 2 — Compare the two approaches

The ratio between the number of policies evaluated by brute-force search and policy iteration is:

$$\frac{N_{\text{BF}}}{N_{\text{PI}}} = \frac{1,048,576}{6} \approx 174.763$$

Thus, in this example, brute-force search evaluates approximately 175.000 times as many policies as policy iteration before reaching the same optimal solution.

Key concept

Policy iteration avoids an exhaustive search over the policy space by using the value of the current policy to systematically construct an improved one. Each policy is therefore selected using information obtained from the previous policy rather than being examined blindly. This is crucial because the number of deterministic policies grows exponentially with the number of states, making brute-force search rapidly impractical as the size of the MDP increases.

0.1.8 Exercise 3.8 — Comparing the Computational Cost of Policy Iteration and Value Iteration

Consider an MDP with 16 states and 4 actions, as in FrozenLake. Suppose that:

- policy iteration requires 5 policy-improvement cycles, with each cycle using an average of 40 policy-evaluation sweeps;
- value iteration requires a total of 250 sweeps to converge.

Answer the following questions:

1. Estimate the total computational cost of policy iteration and value iteration.
2. Determine which algorithm is computationally cheaper under these assumptions.
3. Find the number of value-iteration sweeps M^* for which the total cost of value iteration would equal that of policy iteration, and interpret the result.

Step 1 — Compute the total cost of each algorithm

For policy iteration, one improvement cycle has an approximate computational cost of:

$$C_{PI, \text{cycle}} = O(|\mathcal{S}|^2(L + |\mathcal{A}|))$$

where L accounts for the policy-evaluation sweeps and $|\mathcal{A}|$ for the policy-improvement step. Therefore, over K cycles,

$$C_{PI} = K|\mathcal{S}|^2(L + |\mathcal{A}|) = 5 \cdot 16^2 \cdot (40 + 4) = 56.320$$

For value iteration, each sweep evaluates all actions in every state, giving an approximate cost of

$$C_{VI, \text{sweep}} = O(|\mathcal{S}|^2|\mathcal{A}|).$$

Therefore, over M sweeps,

$$C_{VI} = M|\mathcal{S}|^2|\mathcal{A}| = 250 \cdot 16^2 \cdot 4 = 256.000.$$

Step 2 — Compare the two algorithms*

Since

$$56.320 < 256.000,$$

policy iteration is computationally cheaper in this example. Although each policy-iteration cycle is relatively expensive because it includes multiple policy-evaluation sweeps, only a few cycles are required. In contrast, each value-iteration sweep is cheaper, but the larger number of sweeps results in a higher total computational cost.

Step 3 — Find the break-even point

The two algorithms have the same computational cost when

$$M^*|\mathcal{S}|^2|\mathcal{A}| = K|\mathcal{S}|^2(L + |\mathcal{A}|)$$

Cancelling the common factor $|\mathcal{S}|^2$ and solving for M^* gives:

$$M^* = \frac{K(L + |\mathcal{A}|)}{|\mathcal{A}|} = \frac{5(40 + 4)}{4} = 55$$

Thus, value iteration would be cheaper only if it converged in fewer than 55 sweeps. Since in this example it requires $M=250$ sweeps, policy iteration has the lower total computational cost.

Key concept

Neither algorithm is computationally cheaper in every case. Their relative cost depends not only on the cost of an individual sweep or improvement cycle, but also on how many sweeps or cycles are required to converge for a particular MDP. The break-even point

$$M^* = \frac{K(L + |\mathcal{A}|)}{|\mathcal{A}|}$$

makes this trade-off explicit.

0.1.9 Exercise 3.9 — Discounting and Risk: Shortcut vs. Safe Path

From a state s , the agent must choose between two actions:

- **SHORTCUT**: with probability p , the agent reaches the goal G in one step and receives a reward of $+1$; with probability $(1-p)$, it reaches a terminal hole H and receives a reward of -1 .
- **SAFE**: the agent deterministically reaches an intermediate state with reward 0 and then reaches the goal G one step later with reward $+1$.

Thus, **SHORTCUT** offers an immediate but uncertain outcome, whereas **SAFE** guarantees the positive reward but delays it by one step.

1. Express the state value functions of **SHORTCUT** and **SAFE** as functions of p and the discount factor.
2. Find the success probability for which the agent is indifferent between **SHORTCUT** and **SAFE**.
3. Analyze how this probability changes as the discount factor goes to 0 and as the discount factor goes to 1 . Explain what this reveals about the effect of discounting on the trade-off between a faster but riskier path and a slower but safer one.

Step 1 — Compute the value of each action

For SHORTCUT, the reward is received at the next time step and is therefore not discounted. Its value is the expected immediate reward:

$$v_{\text{SHORTCUT}} = p \cdot (+1) + (1 - p) \cdot (-1) = 2p - 1.$$

For SAFE, the first reward is 0, while the reward of +1 is received one step later and is therefore discounted once:

$$v_{\text{SAFE}} = 0 + \gamma \cdot (+1) = \gamma$$

Step 2 — Find the indifference point

The agent is indifferent between SHORTCUT and SAFE when the two actions have the same expected return. Therefore,

$$v_{\text{SHORTCUT}} = v_{\text{SAFE}} \Rightarrow 2p - 1 = \gamma$$

Solving for p gives the threshold success probability

$$\tilde{p} = \frac{1 + \gamma}{2}$$

Step 3 — Analyze the behavior of the threshold

As $\gamma \rightarrow 0$, the agent becomes increasingly myopic and:

$$\tilde{p} \rightarrow \frac{1}{2}.$$

The delayed reward of the SAFE path is then heavily discounted, so even a SHORTCUT with a success probability only slightly above 0.5 becomes preferable.

As $\gamma \rightarrow 1$, the agent gives increasing weight to future rewards and

$$\tilde{p} \rightarrow 1$$

The certain but delayed reward of the SAFE path is now valued almost fully, so the SHORTCUT becomes preferable only when its probability of success is very close to 1.

Thus, stronger discounting, corresponding to smaller discount factors, lowers the success probability required to choose the faster but riskier path. This illustrates how discounting can shift the optimal policy toward more immediate rewards even when obtaining them involves greater risk.

Key concept

Discounting affects more than the agent's preference for when rewards are received: it can also change which level of risk is optimal. A certain but delayed reward is discounted, whereas an immediate uncertain reward is not. Consequently, decreasing the discount factor can make a faster but riskier action preferable to a slower but safer one. This is the same mechanism illustrated by the FrozenLake discounting experiment.